1. Automation plan

The YAML file defines a **fully automated, authenticated DAST workflow** against the bWAPP application using OWASP ZAP's Automation Framework. It:

- Creates a **context** for the target web applications with include/exclude URL rules.
- Configures **form-based authentication** using the default credentials (bee/bug) at `login.php`.
- Runs an **authenticated traditional spider**, **AJAX spider**, **active scan**, **passive scan wait**, and then generates a **modern HTML report**.

---

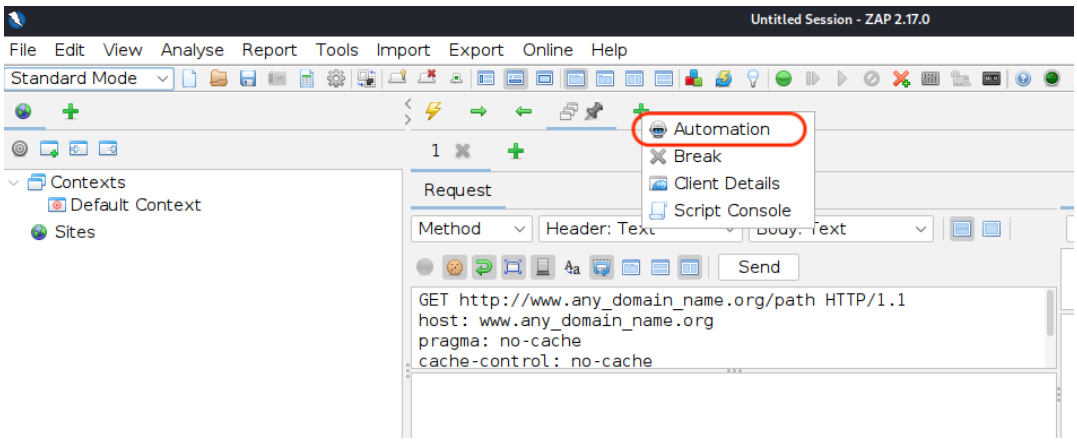
2. Place the YAML file in your ZAP environment

1. Download the `FullScanbWAPPAuth_test_Final.yaml` from the github repository and save it onto the machine running OWASP ZAP. (Link here)
2. Ensure the ZAP instance has network access to the bWAPP host referenced in the plan.

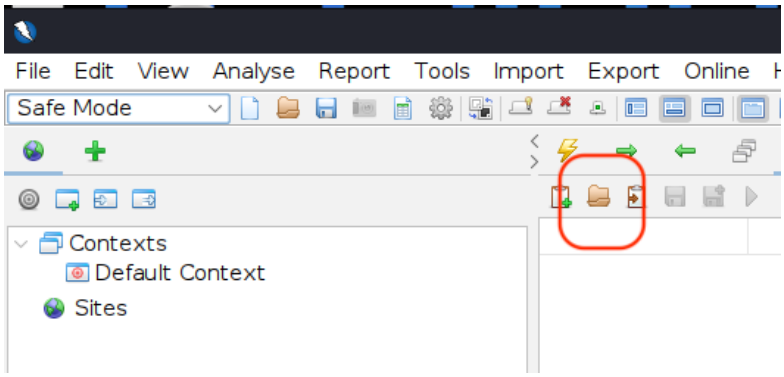
---

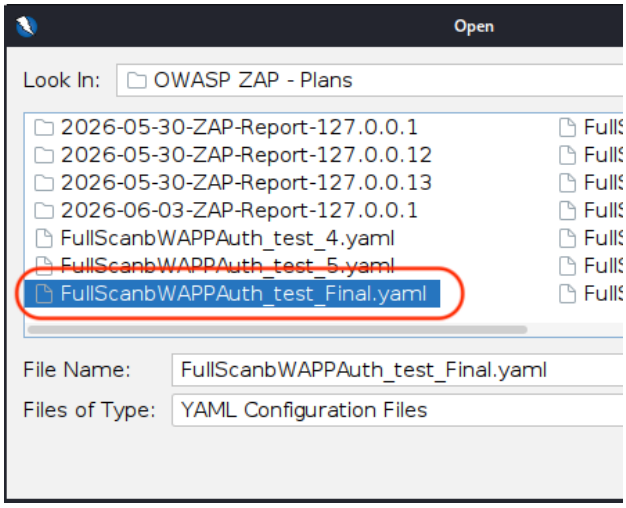
3. Load the plan via the ZAP GUI (Automation tab)

1. Start OWASP ZAP and open the **Automation** tab in the main window.



2. Click **Load plan** (or equivalent) and select the yaml file `FullScanbWAPPAuth_test_Final.yaml`.





3. ZAP will parse the file and display the configured **environment**, **context**, **authentication**, **users**, and **jobs** (spider, spiderAjax, activeScan, passiveScan-wait, report).
4. Review the context name `bWAPP-form`, the base URL `http://bWAPP`, and include/exclude paths to confirm they match your bWAPP deployment (adjust hostnames in the YAML if necessary).

---

4. Verify authentication and user configuration

1. In the Automation view, expand the **env → contexts → bWAPP-form** section:
   - **Authentication method**: `form`.
   - **loginPageUrl / loginRequestUrl**: `http://bwapp/login.php`.
   - **loginRequestBody**: includes `login={%username%}&password={%password%}&security_level=0&form=submit`, which logs in at security level 0 using your test credentials.
2. Confirm the **verification** block:
   - Method `poll`, using `loggedInRegex \QLogout\E` against `http://bwapp/portal.php`, meaning ZAP checks for the string "Logout" to confirm the session is authenticated.
3. Check **sessionManagement**:
   - Method `headers`, with `Cookie: PHPSESSID={%cookie:PHPSESSID%}`, telling ZAP to maintain the PHP session by tracking the `PHPSESSID` cookie.

4. Under **users**, verify the test user:
    - Name: `test`.
    - Credentials: `username: bee`, `password: bug`.

If any of these values differ from your actual bWAPP deployment (hostname, path, credentials), update the YAML and reload it before running the plan.

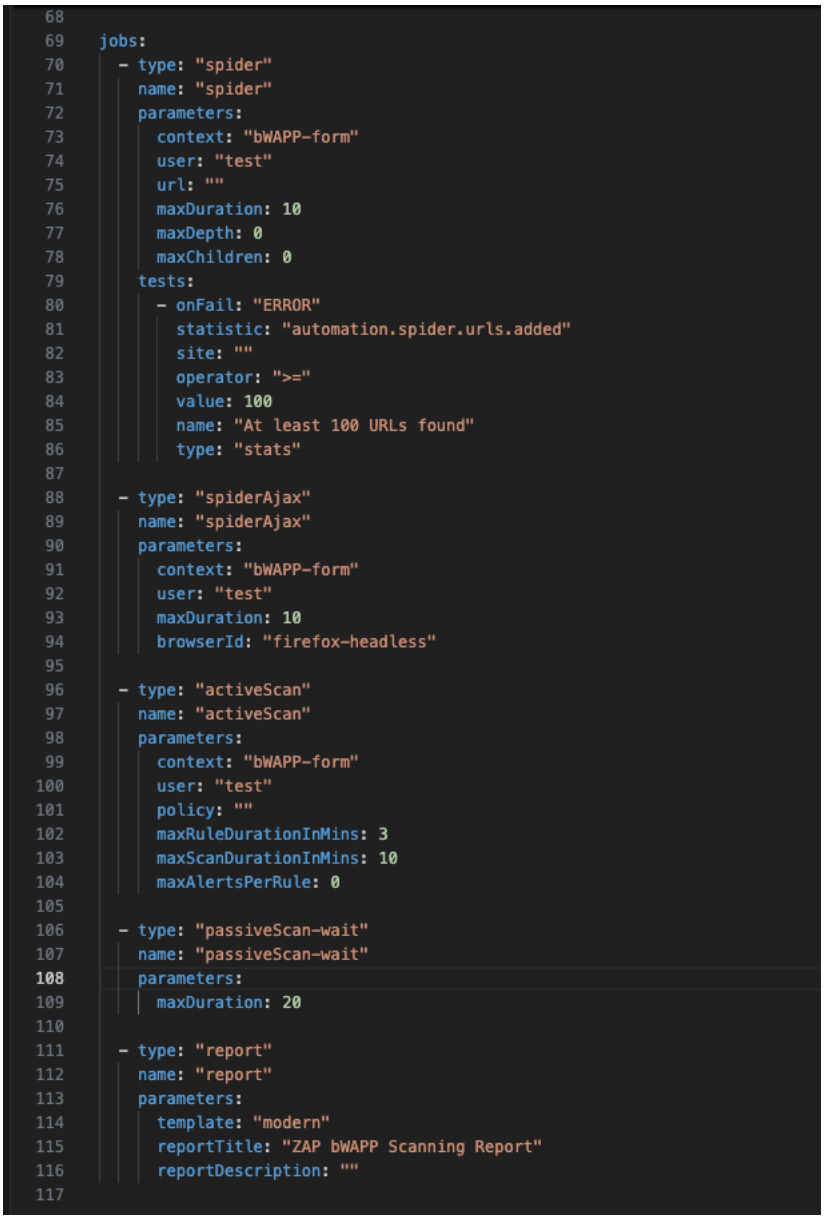
```yaml
env:
  contexts:
    - name: "bWAPP-form"
      urls:
        - "http://bWAPP"
      includePaths:
        - "http://bWAPP.*"
      excludePaths:
        - "http://bwapp/password_change.php"
        - "http://bwapp/user_extra.php"
        - "http://bwapp/security_level_set.php"
        - "http://bwapp/credits.php"
        - "https://itsecgames.blogspot.com/"
        - "http://bwapp/login.php"
        - "http://bWAPP/logout.php"
        - "https://model-hub.mozilla.org/.*"
        #- "http://bwapp/images/.*"
        #- "http://bwapp/fonts/.*"
        #- "http://bwapp/documents/.*"
        #- "http://bwapp/js/.*"
        #- "http://bwapp/stylesheets/.*"
        #- "http://bwapp/passwords/.*"
      authentication:
        method: "form"
        parameters:
          loginPageUrl: "http://bwapp/login.php"
          loginRequestUrl: "http://bwapp/login.php"
          # loginRequestBody: "login={%username%}&password={%password%}&form=submit"
          loginRequestBody: "login={%username%}&password={%password%}&security_level=0&form=submit"
        verification:
          method: "poll"
          loggedInRegex: "\\QLogout\\E"
          pollFrequency: 60
          pollUnits: "seconds"
          pollUrl: "http://bwapp/portal.php"
          pollPostData: ""
      sessionManagement:
        method: "headers"
        parameters:
          Cookie: "PHPSESSID={%cookie:PHPSESSID%}"
      technology:
        exclude: []
        include: []
      users:
        - name: "test"
          credentials:
            username: "bee"
            password: "bug"

  parameters:
    failOnError: true
    failOnWarning: false
    progressToStdout: true
    continueOnFailure: false

  vars: {}
```

---

5. Review scan parameters and limits

In the **jobs** section of the Automation tab, confirm:

1. **Spider job** (`type: spider`):
    - Context: `bWAPP-form`.
    - User: `test` (authenticated spider).
    - `maxDuration: 10` (minutes), `maxDepth: 0` (unbounded depth), `maxChildren: 0` (no per-node limit).
    - Test: requires at least `100` URLs discovered (`automation.spider.urls.added >= 100`), failing the plan if fewer are found.
2. **AJAX spider job** (`type: spiderAjax`):
    - Same context and user.
    - `maxDuration: 10` minutes.
    - `browserId: firefox-headless`.
3. **Active scan job** (`type: activeScan`):
    - Context: `bWAPP-form`, user: `test`.
    - `policy: ""` (default active scan policy).
    - `maxRuleDurationInMins: 3` and `maxScanDurationInMins: 10` to bound runtime per rule and total scan.
    - `maxAlertsPerRule: 0` (no truncation of alerts).
4. **Passive scan wait job** (`type: passiveScan-wait`):
    - `maxDuration: 20` minutes to allow passive analysis to complete.
5. **Report job** (`type: report`):
    - `template: modern`.
    - `reportTitle: "ZAP bWAPP Scanning Report"`.
    - Empty description, which you can later supplement in your dissertation.



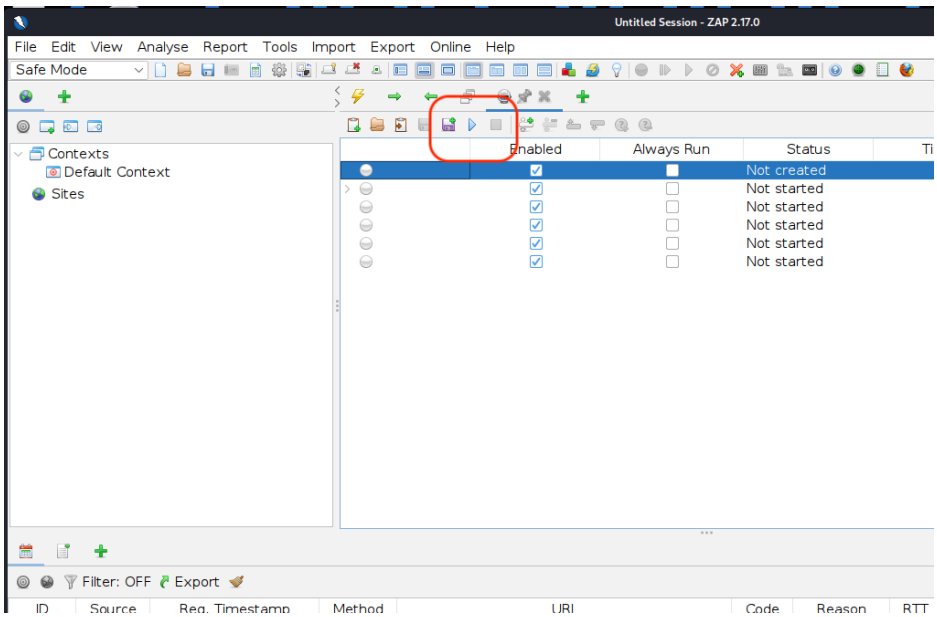
```yaml
jobs:
  - type: "spider"
    name: "spider"
    parameters:
      context: "bWAPP-form"
      user: "test"
      url: ""
      maxDuration: 10
      maxDepth: 0
      maxChildren: 0
    tests:
      - onFail: "ERROR"
        statistic: "automation.spider.urls.added"
        site: ""
        operator: ">="
        value: 100
        name: "At least 100 URLs found"
        type: "stats"

  - type: "spiderAjax"
    name: "spiderAjax"
    parameters:
      context: "bWAPP-form"
      user: "test"
      maxDuration: 10
      browserId: "firefox-headless"

  - type: "activeScan"
    name: "activeScan"
    parameters:
      context: "bWAPP-form"
      user: "test"
      policy: ""
      maxRuleDurationInMins: 3
      maxScanDurationInMins: 10
      maxAlertsPerRule: 0

  - type: "passiveScan-wait"
    name: "passiveScan-wait"
    parameters:
      maxDuration: 20

  - type: "report"
    name: "report"
    parameters:
      template: "modern"
      reportTitle: "ZAP bWAPP Scanning Report"
      reportDescription: ""
```

These settings encode your methodological choices around scan coverage, duration, and reporting for bWAPP.

The parameters in the yaml file will be different for DVWA version.

6. Run the plan from the GUI

1. With the plan loaded and verified, click **Run** in the Automation tab.



2. ZAP will execute the jobs in sequence: authenticated spider → AJAX spider → active scan → passive scan wait → report generation.
3. Monitor progress in the Automation tab or standard output if `progressToStdout: true` (as defined in `parameters`).
4. When the plan completes, locate the generated **HTML report** (Modern template) in the path shown by ZAP and archive it as an artefact for your dissertation.

If `failOnError: true` or `continueOnFailure: false`, any critical error (for example, authentication failure or spider test failure) will stop the plan, which supports reproducible, quality-controlled scans.